ARCSIGHT / OPENTEXT

Writing FlexConnector parsers

A practical guide to regex, tokens, mappings and submessages

English edition | 19 October 2026

CONTENTS

02 / Read this first

03 / 01 / Properties and regex

04 / 02 / A complete first parser

05 / 03 / Tokens, types and time

06 / 04 / Event mapping and severity

07 / 05 / Submessage routing

08 / 06 / A complete submessage parser

09 / 07 / Submessage fields and operations

10 / 08 / Extra processors and multiline

11 / 09 / Windows event case study

12 / 10 / Test and deploy deliberately

13 / 11 / Troubleshooting and handover

14-15 / Sources and verification notes

Includes two complete parsers and raw test logs. Not an official OpenText publication.

Long code lines wrap visually in this PDF. Copy complete examples from the attached properties files; fragments are also available unwrapped in the Markdown edition.

Read this first

This practical handbook focuses on Regex File and Regex Syslog FlexConnector parsers. It is a new tutorial, not a translation of the supplied historical manual. The examples use synthetic events and documentation IP addresses.

What is current - and what is verified

Sources were checked on 19 October 2026. The public SmartConnector CE 25.1 index links a FlexConnector guide labelled CE 24.3, dated July 2024. Newer SmartConnector portal pages also exist; this does not establish a newer revision of the FlexConnector guide. Older HTML chapters are identified as 8.4 and cross-checked against the publicly hosted CE 24.3 PDF. [S1, S2, S13]

Oracle Java SE 21 references explain properties, regex and date-format syntax; they do not assert which JRE your connector ships. Confirm the installed connector version, parser type, and locale before deployment. [S10-S12]

The working model

Stage	Question to answer
Event boundary	Which bytes belong to one event?
Main regex	Which text becomes each capture?
Tokens	What are the names, types and formats?
Submessage	Which ID selects which body pattern?
Normalization	Which event fields receive which values?
Validation	Does the real connector produce the intended event?

Reading route: syntax -> complete example -> types/mappings -> submessages -> advanced processing -> testing. Code blocks marked as fragments must be merged into an existing parser. They are not standalone parsers.

01 / Properties and regex

Treat the raw log, the decoded regex and the .properties value as three distinct representations. A regex tester may accept a single backslash; the property file normally needs it escaped again. Comments use a leading # or !. A trailing unescaped backslash continues a property onto the next physical line. Do not add it to every line. [S10]

Intent	Decoded regex	In .properties
Digit	\d	\\d
Tab	\t	\\t
Literal dot	\.	\\.
Capture a field	([^\t]*)	([^\\t]*)

A deliberate tab-delimited example

```
regex=([^\\t]*)\\t([^\\t]*)\\t([^\\t]*)
```

This original example describes exactly three fields. The * allows an empty field; replacing it with + rejects an empty field. Preserve consecutive tabs. A visual gap is not proof of a tab: U+2003 EM SPACE differs from U+0009 TAB. Load the original file rather than copying a formatted chat transcript.

Capture only what you need

Use (...) to extract a token and (?:...) to group structure without adding a token. Escape dots in IP-shaped patterns. Avoid nested unbounded repetition such as (.*)+. An address-shaped regex is not validation of the numeric IPv4 range. [S11]

```
# Structural IPv4 shape, not octet-range validation
regex=((?:\\d{1,3}\\.){3}\\d{1,3})
```

Authoring practice: write down the intended fields first, then build the regex from left to right. Test the entire event. A substring match in a generic tester is insufficient evidence that the parser is correct.

02 / A complete first parser

Save this original example as training-basic.sdkrfilereader.properties. It expects the exact key order shown below. Configure the source time zone separately because this timestamp carries no offset.

```
2026-10-19 10:02:15 level=WARN user=alice src=192.0.2.10 action=LOGIN

# Training example: one line = one event
regex=(\\S+ \\S+) level=(\\w+) user=(\\S+) src=(\\S+) action=(\\w+)
token.count=5
token[0].name=EventTime
token[0].type=TimeStamp
token[0].format=yyyy-MM-dd HH:mm:ss
token[1].name=Level
token[1].type=String
token[2].name=User
token[2].type=String
token[3].name=SourceIP
token[3].type=IPAddress
token[4].name=Action
token[4].type=String

event.deviceVendor=__stringConstant("ExampleVendor")
event.deviceProduct=__stringConstant("TrainingApp")
event.name=Action
event.endTime=EventTime
event.sourceUserName=User
event.sourceAddress=SourceIP
event.deviceAction=Action
event.deviceSeverity=Level

severity.map.veryhigh.if.deviceSeverity=CRITICAL
severity.map.high.if.deviceSeverity=ERROR
severity.map.medium.if.deviceSeverity=WARN
severity.map.low.if.deviceSeverity=INFO,DEBUG
```

Expected result: User=alice, SourceIP=192.0.2.10, Action=LOGIN, deviceSeverity=WARN. The example policy maps WARN to medium. Syntax and captures are locally checked; ArcSight conversion and destination behavior still require a connector test. [S4-S6]

03 / Tokens, types and time

Token indices start at zero. For ordinary numbered capture groups, token[0] corresponds to group 1. Keep token.count consistent with the capture count; noncapturing groups do not consume a token. A name is a reference used later in mappings. [S4]

Type / use	Practical check
String / IDs, names, messages	Preserve identifiers such as 001 rather than converting them.
Integer / ports, small counts	Verify range and empty-field behavior.
Long / large counters	Avoid overflow in long-running counters.
IPAddress / network address	Use a hostname field for a DNS name, not sourceAddress.
TimeStamp / event time	Supply a matching format and verify source time zone.

Timestamp example

```
token[0].type=TimeStamp
token[0].format=EEE MMM dd HH:mm:ss yyyy
```

For Thu Sep 03 10:02:15 2026, EEE and MMM require a locale that recognizes English names. MM means month; mm means minute. HH is a 24-hour clock. yyyy is the calendar year; YYYY is a week-based year and can produce surprises around New Year. Quote fixed text inside a date format with single quotes. [S12]

An empty token is a result, not necessarily an error

For A<TAB><TAB>B, the second field is empty. A captured empty field and an optional group that did not participate are different cases. Never shift later token names to conceal an empty field. Compare each expected field against the raw separator positions.

The type list is a working subset, not an exhaustive catalog. Check the built-in token types and event-field tables in the public CE 24.3 guide before mapping a new type. [S2]

04 / Event mapping and severity

Capturing text does not yet normalize an event. A mapping assigns a token or operation result to a schema field; compatible types matter. The device that reports an event is not necessarily the source or destination described by it. [S5]

```
event.sourceUserName=User
event.sourceAddress=SourceIP
event.deviceHostName=ReportingHost
event.deviceCustomString1=Channel
event.deviceCustomString1Label=__stringConstant("Channel")
```

Fragment only: User, SourceIP, ReportingHost and Channel must already be declared. __stringConstant creates a literal; a bare token name references a parsed value.

Choose fields from the event meaning

Authoring exercise: in a firewall denial, write down who initiated the connection, who received it, and which device logged it. Then map those roles. Do not assign the first address you find to sourceAddress without checking its meaning. For Windows examples, keep provider/channel and event ID available so downstream users can distinguish event families.

Normalize severity deliberately

Set event.deviceSeverity from the original level, then define the intended normalized level. Supported severity rule suffixes are low, medium, high and veryhigh. Values can be lists or numeric ranges. Do not assume that a larger vendor number means greater severity. [S6]

```
event.deviceSeverity=Level
severity.map.high.if.deviceSeverity=ERROR
severity.map.medium.if.deviceSeverity=WARN
severity.map.low.if.deviceSeverity=INFO,DEBUG
```

This is a sample policy, not a universal Windows severity policy. Include every observed value in your test corpus and review unmapped values. Keep rules non-overlapping so their meaning remains easy to audit.

05 / Submessage routing

submessage.token identifies the body token. submessage.messageid.token identifies the routing token. A definition has one documented messageid value; it is not a friendly label. After routing, its patterns are tried in order and the first successful pattern wins. A definition without messageid is the default for otherwise undefined IDs. [S3]

Configuration	Meaning
submessage.token=Message	Parse the captured Message value.
submessage.messageid.token=EventId	Use EventId to select a definition.
submessage[0].messageid=136	Select this definition for the ID value 136.
submessage[0].pattern.count=2	Try pattern[0], then pattern[1] if needed.

ID 1 and ID 2 share a body format

The worked example on the next page repeats the same pattern under two IDs. This is intentionally explicit. Do not treat messageid=1,2 or messageid=1|2 as documented multi-ID syntax: support was not established in the reviewed sources.

A default definition can share a regex across otherwise unhandled IDs, but it also accepts other unknown IDs. It is not an allowlist for only 1 and 2. The example retains a declared routing token; it does not rely on omitting the global messageid token setting.

A diagnostic question before editing regex

If Message contains a full sentence and the routing token is Message, compare that entire value with the configured ID. A definition called Var1 will not be selected for a DROP sentence. If no route exists, changing its body regex cannot fix the routing decision.

The editor can preview an unselected pattern independently. That preview is a debugging aid, not proof that the connector chose it. Treat actual route and selected-pattern match as separate results.

06 / A complete submessage parser

Save as training-submessages.sdkrfilereader.properties. This separate parser is used with training-submessages.log; do not append it to the previous complete parser.

```
# Training example: exact IDs and one default
regex=id=(\\d+) body=(.*)
token.count=2
token[0].name=EventId
token[0].type=String
token[1].name=Message
token[1].type=String
event.deviceVendor=__stringConstant("ExampleVendor")
event.deviceProduct=__stringConstant("TrainingApp")
event.name=__stringConstant("Account activity")
event.deviceEventClassId=EventId
event.message=Message
submessage.messageid.token=EventId
submessage.token=Message
submessage.count=3

submessage[0].messageid=1
submessage[0].pattern.count=2
submessage[0].pattern[0].regex=user=(\\S+) src=(\\S+)
submessage[0].pattern[0].fields=event.sourceUserName,event.sourceAddress
submessage[0].pattern[0].types=String,IPAddress
submessage[0].pattern[1].regex=(anonymous)
submessage[0].pattern[1].fields=event.message

submessage[1].messageid=2
submessage[1].pattern.count=1
submessage[1].pattern[0].regex=user=(\\S+) src=(\\S+)
submessage[1].pattern[0].fields=event.sourceUserName,event.sourceAddress
submessage[1].pattern[0].types=String,IPAddress

# Default: intentionally no submessage[2].messageid
submessage[2].pattern.count=1
submessage[2].pattern[0].regex=(.*)
submessage[2].pattern[0].fields=event.message
submessage[2].pattern[0].extramappings=event.name=__stringConstant("Unclassified")
```

The last sample, id=1 body=unexpected format, deliberately fails the known-ID patterns. Unknown ID 9 uses the default. This distinction is part of the test, not a defect.

07 / Submessage fields and operations

Submessage captures are referenced as $1, $2 and so on. fields assigns event destinations; types and formats describe conversion. mappings can provide operations instead of direct capture assignments, using a pipe separator by default. [S2, section Using Sub-Messages]

```
# Fragment: three captures; map address, port, then message
submessage[0].pattern[0].regex=(.*) Sender: (\\S+):(\\d+)
submessage[0].pattern[0].fields=event.sourceAddress,event.sourcePort,event.message
submessage[0].pattern[0].types=String,IPAddress,Integer
submessage[0].pattern[0].mappings=$2|$3|$1
```

Here capture 1 is text, capture 2 is an address and capture 3 is a port. The types describe those captures; mappings rearranges their assignment to the listed destination fields. If no rearrangement is needed, keep capture order and field order aligned and omit mappings.

Keep transformations visible in tests

A regex preview proves extraction only. It does not prove that an address converts successfully, an operation returns the expected value, or a later mapping preserves it. Write separate expected values for raw captures and normalized fields. Test malformed addresses and ports outside the intended domain.

Avoid a catch-all hiding a specific pattern

Original exercise: place (.*) before a DROP-specific pattern. Observe that the generic pattern consumes the message first. Restore the specific pattern before the fallback and retest. Record both the route index and extracted fields; a green match alone does not prove the intended alternative won.

Do not confuse a properties escape, a regex capture number and an ArcSight operation argument. Store the editable properties file alongside test logs so reviewers can verify every layer.

08 / Extra processors and multiline

Extra processors link parsing or enrichment stages. Their input is selected by field, typically an event field, rather than by submessage.token. Choose this mechanism when one event mixes formats, for example a header and a structured payload. [S7]

```
# Fragment; requires a separately deployed child parser
extraprocessor.count=1
extraprocessor[0].type=regex
extraprocessor[0].filename=training/body
extraprocessor[0].field=event.message
extraprocessor[0].flexagent=true
extraprocessor[0].clearfieldafterparsing=false
```

This fragment is not a complete deployment: training/body must resolve to the linked parser for your connector. Review field-clearing and mapping-override behavior before chaining stages. Document each input field, output field and failure case. [S7]

Multiline is event framing

For supported Regex Log File readers, define an event start and optionally an end. Lines are joined before parsing; the documented default join delimiter is a space. multiline.max.count limits growth. A single-line regex test does not test this reader behavior. [S8]

```
# Fragment: start of a new timestamped event
multiline.starts.regex=\\d{4}-\\d{2}-\\d{2} .*
multiline.max.count=100
```

Authoring practice: test the last event at end-of-file, delayed continuation lines and a missing terminator. Never solve an event-boundary problem solely by making the main regex more permissive.

09 / Windows event case study

Consider a 15-field MSWinEventLog record. In the project sample, token[12] is Something (group 13), token[13] is Message (group 14), and token[14] is Size. The supplied DROP event has an empty Something field but a nonempty Message field. These names are parser decisions, not universal Windows token names.

```
[DROP] Dropping received RCP messages from 192.0.2.10:3333.
```

Inspection	Expected observation
Group 13 / token[12]	Empty value, bounded by consecutive tabs
Group 14 / token[13]	The complete DROP sentence
Info token	Netft Trace Message, including spaces
Event ID token	136, if captured from the supplied field

Two distinct debugging failures

First, a copied line containing EM SPACE pairs fails a tab-specific main regex. Second, a correct body still fails routing if the selected ID token is Message but the configured ID is Var1. Fix separators and routing independently; do not replace tabs automatically in production because empty columns have meaning.

A body-only DROP fragment

```
submessage[0].pattern[0].regex=\\[DROP\\] Dropping received RCP messages from ((?:\\d{1,3}\\
  .){3}\\d{1,3}):(\\d{1,5})\\.
submessage[0].pattern[0].fields=event.sourceAddress,event.sourcePort
submessage[0].pattern[0].types=IPAddress,Integer
```

This fragment has two captures; it does not include the outer entire-message capture used in some earlier examples. Update fields and types whenever the capture layout changes. Apply it only inside the intended ID/default definition.

10 / Test and deploy deliberately

The following workflow is an authoring recommendation, not a vendor certification procedure. Keep the parser, raw fixtures, expected results and connector version together in version control.

Test layer	Evidence to retain
Properties	Decoded regex; exact token names and counts
Regex	Complete matches and expected failures
Submessage	ID value, selected definition/pattern, raw captures
Conversion	Actual typed addresses, ports and timestamps
Normalization	Expected event fields and severity
Reader / destination	Boundaries, encoding, timestamps, delivered events

Expected results for the bundled submessage log

Line	Expected outcome
1	ID 1 -> submessage[0].pattern[0]
2	ID 1 -> submessage[0].pattern[1]
3	ID 2 -> submessage[1].pattern[0]
4	Unknown ID 9 -> default submessage[2]
5	Known ID 1, wrong body -> pattern mismatch

Add negative samples: missing separators, truncated bodies, empty fields, unexpected IDs, long messages, non-ASCII names, changed key order and invalid timestamps. Include examples not used while writing the regex.

Deployment checklist

Confirm the selected connector type and its naming/location rules. The CE 24.3 guide documents custom parsers under current/user/agent/flexagent and specific syslog naming under its syslog subdirectory. Do not treat a File Reader parser as a drop-in Syslog parser. Back up the current parser; deploy in a test connector, inspect agent.log, and compare destination events before promotion. [S2, installation and custom syslog chapters]

Local sample validation checks extraction and routing only. Neither a partial match nor a successful .NET preview guarantees Java runtime conversion, operations, multiline framing or destination delivery.

11 / Troubleshooting and handover

Symptom	First investigation
Main regex mismatch	Raw separators, encoding, framing and properties escapes
Submessage input is empty	Is the main match complete? Was the chosen group empty?
No matching ID / default	Compare the captured ID with configured values, including spaces.
Selected preview matches; batch fails	Independent preview may not be the actual route.
Wrong fields despite a match	Capture numbering, mappings order and semantic roles
Correct text, wrong timestamp	Date format, locale, time zone and calendar year
One pattern matches everything	Move generic patterns after specific ones.
Works locally, fails in connector	Java behavior, type conversion, operations and reader settings

What to put in a parser repository

Include the properties file, synthetic/anonymized raw logs, expected routes and fields, installation notes, connector build, and a change log. State which checks were local and which ran on a real SmartConnector. Keep secrets and personal data out of examples.

Boundaries of this handbook

This is a practical Regex parser guide. JSON/XML/database/Kafka/REST collection, full operation catalogs, categorization taxonomies and connector administration require their dedicated manuals. Unsupported or unverified syntax is not invented to make an example shorter.

The attached examples are original training configurations. Their expected capture and route behavior is checked with the project ParserCore. That engine uses .NET; a SmartConnector acceptance test remains necessary. Keep the supplied editable sources so examples can be adapted without copying code from a PDF.

Mini glossary: capture = matched group; token = named typed value; mapping = assignment to a normalized field; message ID = routing value; pattern = one body regex; default = route for otherwise undefined IDs; extra processor = linked processing stage.

Sources / reference register

All links checked on 2026-10-19. Source labels in the text refer to this register. Older HTML is retained as a directly linkable reference, not presented as a new edition.

S1 / OpenText: SmartConnector CE 25.1 documentation index

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-25.1/

S2 / OpenText: Developer's Guide to ArcSight FlexConnectors, CE 24.3 (244 pages)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-25.1/pdfdoc/flexconn_devguide/flexconn_devguide.pdf

S3 / OpenText: Using Sub-Messages for Multiple File Formats (8.4)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-8.4/flexconn_devguide/Content/convertFlex/Sub_Messages.htm

S4 / OpenText: Token Declarations (8.4)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-8.4/flexconn_devguide/Content/convertFlex/TokenDeclarations.htm

S5 / OpenText: Event Mapping (8.4)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-8.4/flexconn_devguide/Content/convertFlex/Event-Mapping.htm

S6 / OpenText: Severity Mapping (8.4)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-8.4/flexconn_devguide/Content/convertFlex/SeverityMapping.htm

S7 / OpenText: Extra Processors (8.4)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-8.4/flexconn_devguide/Content/convertFlex/ExtraProcessors.htm

Sources / reference register

All links checked on 2026-10-19. Source labels in the text refer to this register. Older HTML is retained as a directly linkable reference, not presented as a new edition.

S8 / OpenText: Supported Regular Expressions / Multi-line Parsing (8.4)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-8.4/flexconn_devguide/Content/convertFlex/Regular_Expressions.htm

S9 / OpenText: Parser Properties for all Regex FlexConnectors (8.4)

https://www.microfocus.com/documentation/arcsight/arcsight-smartconnectors-8.4/flexconn_devguide/Content/convertFlex/ConfigProperties-for-all-RegexFlexConnectors.htm

S10 / Oracle: java.util.Properties, Java SE 21

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Properties.html

S11 / Oracle: java.util.regex.Pattern, Java SE 21

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/regex/Pattern.html

S12 / Oracle: java.text.SimpleDateFormat, Java SE 21

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/text/SimpleDateFormat.html

S13 / OpenText: SIEM SmartConnector 26.1 portal (version context only)

https://docs.microfocus.com/doc/2097/26.1/home